

Chapitre 1 : Histoire de l'informatique

1.1 Des nombres et des hommes

Comptage : quantifier ses propres richesses.

Basé sur une technique très simple : à un **nombre** donné correspondaient un nombre équivalent d'éléments physiques.

Chiffre : représentation symbolique d'un nombre.

Numération de position : permet de représenter de très grands nombres d'une manière compacte.

⇒ A ça vient s'ajouter le **zéro**, essentiel pour représenter une absence de valeur
exemple : 12, 120, 102, ...

Résumé des systèmes de numération les plus connus :

Base	Nom	Usage	Origine
1	unaire	comptage	/
2	binaire	Logique, électronique, informatique	/
5	quinaire	/	Aztèque
7	septénaire	Jours de la semaine, notes (ton)	/
8	octal	Informatique	Premiers ordinateurs
10	décimal	Système de numération le + répandu	Chinois
12	duodécimal	Mois, heures, musiques..	Egyptiens
16	hexadécimal	Informatique	/
20	vicésimal	/	Mayas
60	sexagésimal	Trigonométrie, minutes, ...	Babyloniens, indiens, arabes, ...

1.2 Deux millénaires de progrès

John Neper : invente la théorie des **logarithmes**.

Wilhelm Schickel : construit une **machine à calculer** sur base des **bâtons de Neper**.

Blaise Pascal : invente une **machine à calculer (Pascaline)**, permettant de + et - deux nombres à 6 chiffres (manque de précision).

Gottfried Wilhelm Leibniz : **améliore la Pascaline** en y ajoutant les " X " et " -/- ". Il invente le **système binaire et l'arithmétique binaire**; base des ordinateurs actuels.

Joseph jacquard : invente le **premier système de programmation via un système de bandes de carton perforées** pour créer à répétition des motifs dans la pression de tisseur (ex : pull).

Charles Babbage : invente une machine à différence pour tables de calcul puis invente la **machine analytique** qui permet de réaliser des opérations en fonction d'un programme établi sur des cartes perforées.

George Boole : invente les bases mathématiques de la logique moderne. **L'algèbre de Boole** est la base de la conception des circuits électroniques digitaux.

Herman Hollerith : invente un **calculateur de stat. à cartes perforées** pour recenser la population 🇺🇸. En 1896 il fonde la Tabulating Machine Company qui deviendra **IBM** en 1924.

John Albrose Fleming : invente le **premier tube à vide** (permet la création des premiers ordinateurs électroniques).

Alan Turing : crée la **machine de Turing**. Il est à l'origine de la formalisation des concepts d'algorithme et de calculabilité.
⇒ **Père fondateur de l'algorithme**.

Claude Shannon : **père fondateur de la théorie de l'information**.

Konrad Zuse : construit les **premiers calculateurs électromécaniques (Z1 Et Z2)** basé sur le système binaire.

Howard Aiken : Invente un **calculateur électromécanique**, le Mark I.

1.3 Evolution des ordinateur

Première génération

Composants : relais, **tubes à vides**, résistances.

Logiciels : langage machine.

Deuxième génération

Composants : transistors, mémoire à tores de ferrite, imprimantes, bandes magnétiques.

Logiciels : apparition de système d'exploitation, langages évolués FORTRAN, COBOL.

Troisième génération

Composants : circuits intégrés (faible consommation, fiables, encombrement réduit).

- De nouvelles fonctionnalités apparaissent : multiprocesseur, système d'exploitation multitâches, accès interactif, réseaux.

Logiciels : /

Quatrième génération

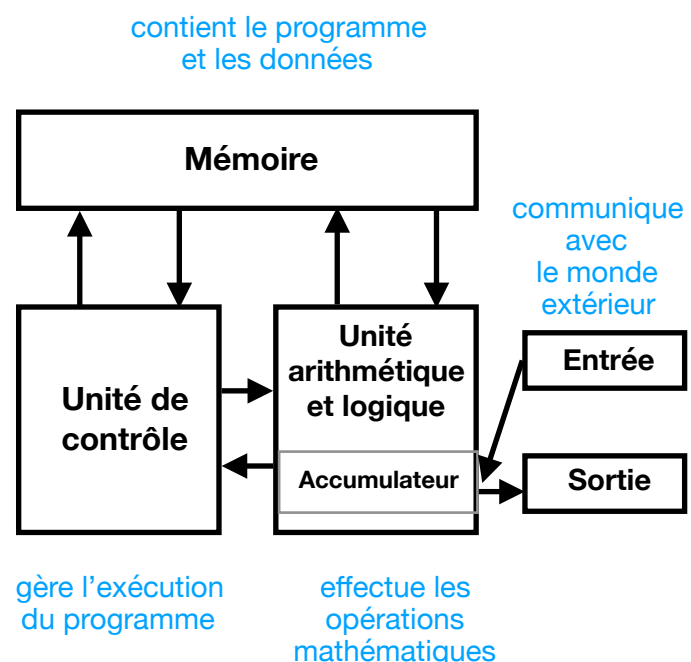
Composants : microprocesseurs, intégration à très large échelle (VLSI).

- Apparition de la miniaturisation (+ de composant car + petit => ordi + puissant).
- Apparition de l'ordinateur personnel (PC).
- Périphériques de grande capacité.
- Convergence numérique : début du monde digital.

Logiciels : /

La machine de Von Neumann :

- Machine universelle contrôlée par un programme.
- Instructions codées en binaire et stockées en mémoire.
- Le programme peut modifier ses propres instructions.
- Exécution des instructions en séquence.



L'architecture ou machine de Von Neumann constitue une véritable **révolution** dans la manière de concevoir un ordinateur car elle propose une architecture qui est très facilement programmable. L'unité de contrôle détermine la logique d'exécution d'un programme. Le programme est quant à lui défini par une séquence d'instruction stockées en mémoire sous la forme d'un code binaire.

1.4 Evolution des performances

Loi de Moore : le nombre de transistors sur une puce double tous les 18 mois.

Avantages / Inconvénients de l'intégration à large échelle des composants :

Avantages	Inconvénients
Augmentation de la capacité des puces mémoires	Impossible de réduire indéfiniment la taille d'un transistor
Augmentation de la vitesse des composants => augmentation de la fréquence d'horloge	Limites atomiques bientôt atteintes
Processeurs + sophistiqués et performants	Densité trop forte => phénomène parasite de fuite de courant
Plusieurs composants sur 1 puce	Quantité d'énergie dissipée devient problématique

Solutions envisageables :

Changement radical de technologie :

- Utiliser l'arséniure de gallium => changement de matériaux.
- Supraconducteurs => résistance nulle.
- Composants optiques => augmentation vitesse de transition.
- Dispositif quantiques (technologie trop complexe).

A plus court terme :

- Architecture multiprocesseurs (plusieurs cœurs ou processeurs interconnectés).
- Architectures parallèles distribuées (fermes de serveurs).
- Programmation parallèle efficace.

Chapitre 2 : Pour Présentation générale d'un ordinateur

2.1 Qu'est-ce qu'un ordinateur ?

Ordinateur : machine automatique de traitement de l'information, obéissant à des programmes formées par des suites d'opérations arithmétiques et logiques. Un système informatique est donc composé d'une *partie matérielle* et d'une *partie logicielle*.

Quelques éléments au sein d'un ordinateur :

- L'unité centrale qui contient :
 - Le processeur
 - La mémoire principale
 - Les gestionnaires d'entrées/sorties
- Des périphériques d'entrée
 - clavier, souris, scanner, ...
- Des périphériques de sortie
 - écran, imprimante, haut-parleur, ...
- Des périphériques de stockage
 - Disques durs
 - Lecteur / graveur
 - Clé USB
- Des périphériques de communication
 - Carte réseau
 - Modem

Différentes gammes d'ordinateur :

- Ordinateur **jetable** : souvent de simple puce aux fonctionnalités très limitées ne coûtant que 1€. Exemple : une carte musicale
- Ordinateur **embarqué** (microcontrôleurs) : ordinateur programmable mais aux fonctionnalités limitées ne coûtant que quelques €. Il est associé à d'autres appareils dans le but de les piloter et souvent d'assurer l'interface avec l'utilisateur.
Exemple : Electroménagers , appareils électroniques divers.
- **Smartphones** et **consoles** de jeux : correspond au concept plus traditionnel de l'ordinateur, l'accent est mis sur une fonctionnalité bien précise, ces systèmes ne sont pas prévus pour évoluer, coûtant quelques dizaines voir centaines d'€.
Exemple : Xbox One, iPhone, ...
- Ordinateur **personnel (PC)** : c'est l'ordinateur traditionnel coûtant quelques centaines ou milliers d'€. Il n'est utilisé que par un utilisateur à la fois.

- **Serveurs** : version « gonflée » d'un ordinateur personnel, est destiné à être connecté à un réseau afin de fournir des services à distance à des utilisateurs. Ses composants sont plus puissants et ont une plus grande capacité qu'un pc et n'a pas nécessairement une interface utilisateur. Il coûte quelques milliers voir centaines de milliers d'€.
- **Ferme de serveur (cluster)** : plusieurs serveurs standards connectés entre eux par un réseau ultra-rapide. Ils sont équipés d'un logiciel pour répartir la charge et traiter énormément de demande simultanément. Parfois utilisé pour une collaboration commune à la résolution d'un problème unique.
- **Superordinateurs, mainframes** : en voie de disparition, ils ont une architecture spécifique et révolue, on peut encore en trouver dans le secteur bancaire. Les cluster coûtent maintenant beaucoup moins cher et sont aussi puissants.

« L'informatique dans les nuages » ou le **cloud** :

Définition : un « cloud » est une ou plusieurs fermes de serveurs qui fournissent des services aux utilisateurs connectés à travers internet.

Intérêt du cloud :

- Mutualiser les ressources pour fournir à moindre coût un service.
- Accessible de n'importe où (internet) => s'affranchir des limites géographiques.
- Stockage de données permettant la synchronisation d'appareils, partage de documents, gestion collaborative de document.

Service cloud pour les entreprises :

- **IaaS - Infrastructure as a Service** :

Location de l'**infrastructure matérielle** à une entreprise (connectivité réseau, serveurs, stockage). Le fournisseur de service est responsable de l'installation, la configuration de base et de la maintenance. L'entreprise est responsable de la partie **applicative**. L'offre IaaS permet une grande **flexibilité** aux entreprises.

- **PaaS - Plateforme as a Service** :

Inclut l'infrastructure mais en plus le **système d'exploitation** et les **logiciels de base** (B.D, serv. Web, outils de développement, ...). permettant à l'entreprise de développer et déployer rapidement **ses** propres applications.

- **SaaS - Software as a Service** :

Le gestionnaire du cloud offre un service applicatif et est donc **responsable** de l'intégralité de la **solution** (connectivité, matériel, logiciel). Le client final peut uniquement adapter la solution finale à ses besoins.

Questions préoccupantes à propos du cloud sur :

- La sécurité et confidentialité des données stockées.
- La disponibilité de l'accès à Internet.
- La disponibilité du service.
- La dépendance forte à l'égard d'un fournisseur.

2.1 Organisation interne d'un ordinateur monoprocesseur

La plupart des structures internes des ordinateurs monoprocesseurs suivent le principe d'organisation de la machine de [Von Neumann](#).

On y retrouve :

- L'unité centrale de traitement (**CPU**) :
 - l'unité de contrôle
 - l'unité arithmétique et logique (ALU)
 - un ensemble de registre
- La mémoire centrale
- Les périphériques d'entrées et sorties.

L'unité centrale de traitement :

Le CPU est le centre névralgique de l'ordinateur. C'est lui qui [exécute les instructions d'un programme](#). Le programme à exécuter et ses données sont d'abord chargés en [mémoire centrale](#). La répétition de cette séquence d'opération s'appelle un [cycle d'exécution](#).

1. Lire une instruction en mémoire.
2. Décoder cette instruction pour déterminer l'opération à exécuter.
3. Obtenir les données utilisées pour l'opération.
4. Réaliser le traitement approprié via l'ALU.
5. Sauvegarder le résultat éventuel.
6. Passer à l'instruction suivante.

L'[unité de contrôle](#) assure le séquençage des opérations au sein du CPU. L'ALU réalise les opérations arithmétiques et logiques demandées par le programme. Les données utilisées par l'ALU se trouvent soit en [registre](#) soit en [mémoire centrale](#).

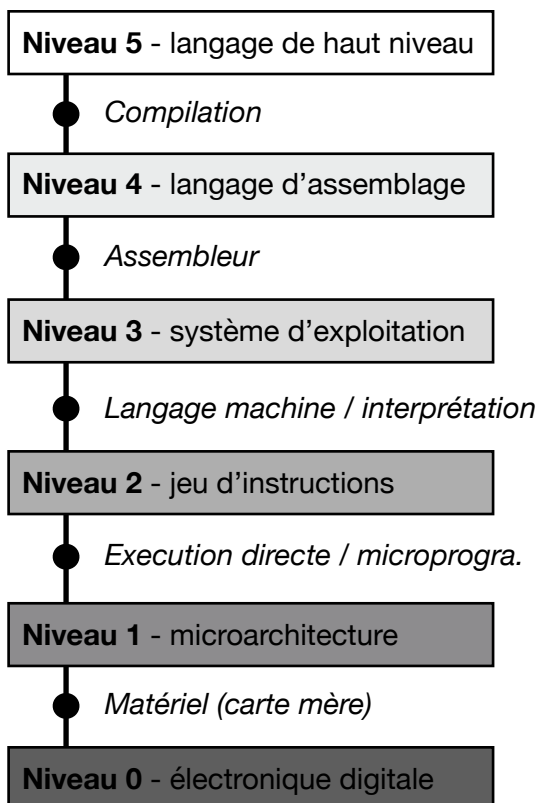
La mémoire centrale :

Elle contient les instructions du programme et les données manipulées, toutes les deux codées en binaire. Le temps pour accéder/écrire à une donnée en mém. centrale est très long donc il faut utiliser au maximum les **registres du processeur**. Afin de gagner en rapidité, on utilise aussi une mémoire intermédiaire plus petite mais beaucoup plus rapide appelée **mémoire cache** ou **antémémoire**.

Les entrées et sorties :

La vitesse de transfert de données entre le CPU et les périph. e/s est très faible donc le CPU ne pilote pas le périphérique mais fait appel à un contrôleur spécialisé sur le bus auquel il délègue la tâche. Un ordinateur contient au moins deux bus : un pour le système qui fournit un accès très rapide à la mémoire au CPU et l'autre, un bus d'entrées/sorties servant à connecter les contrôleurs d' e/s.

2.1 Architecture en couche



Niveau 0 : niveau le plus bas de l'ordinateur, il est composé des **éléments électroniques** de base (portes logiques, bascules, horloges, ...) servant à réaliser des opérations logiques, construire des registre, ...

Niveau 1 : **organise** et **connecte** les éléments de base. La microarchitecture regroupe l'unité de contrôle, la mémoire centrale, la mémoire cache, les bus de communication. Ces éléments matériels définissent les fonctionnalités offertes aux couches supérieures.

Niveau 2 : **spécification formelle** des instructions machines offertes par les niv. 1 et 2; il décrit les instructions que la microarchitecture est capable d'exécuter. Le code machine est le seul langage de programmation qu'un ordinateur peut comprendre.

Niveau 3 : responsable de la **gestion des ressources matérielles** de l'ordinateur. L'OS peut enrichir le jeu d'instruction de base par des nouvelles qui seront interprétés par le microprogramme. Une partie de l'OS est stocké dans une mémoire permanente, c'est le **micrologiciel (firmware)**. Sa première tâche est de charger en mémoire le reste du s.e. Tous les systèmes d'exploitation contiennent un **noyau (kernel)** qui fournit des fonctionnalité de base. Ce niveau 3 est la frontière entre la partie **système** et la partie **utilisateur** de l'ordinateur.

Niveau 4 : le langage d'assemblage est une représentation symbolique du code machine, plus facile à utiliser par l'être humain. Un programme ne peut directement être exécuté par le processeur, il doit d'abord être traduit en code machine via un **assembleur**.

Niveau 5 : les langages de haut niveau (C, C#, java, PHP, ...) sont les langages utilisés par les programmeurs pour résoudre un problème. Ces langages proposent des constructions plus évoluées et une facilité d'utilisation plus grande que l'assembleur.

Compilation : traduction d'un programme écrit dans un langage de haut niveau vers un programme équivalent en code machine. La compilation est effectuée avant l'exécution du programme via un **compilateur**.

Interprétation : l'interprétation d'un langage de haut niveau consiste à utiliser un **interpréteur** (logiciel spécial) qui va analyser, traduire et exécuter au fur et à mesure les instructions du programme.

2.4 Architecture CISC et RISC

Le jeu d'instructions offert par un processeur peut être plus moins riche suivant le processeur, ayant un compris entre deux objectif :

- facilite la vie du programmeur via un jeu d'instructions riches.
- garder une architecture matérielle simple et efficace.

CISC : *Complex Instruction Set Computer* - « ordinateur à jeu d'instructions complexe »

Instruction très commode à utiliser mais :

- Mise en oeuvre des instructions rend le processeur très complexe.
- La complexité du processeur ralentit l'exécution des instructions simples.
- De nombreux logiciels ne sont pas optimisés.

RISC : *Reduced Instruction Set Computer* - « ordinateur à jeu d'instructions réduit »

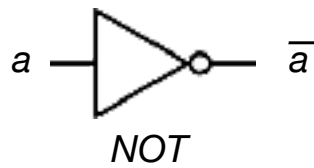
- Propose un jeu d'instructions limité à un nombre d'opération simple.
- Ces instructions ont une taille fixe.
- La mise en œuvre des instructions au niveau matériel est grandement facilitée, ce qui permet une exécution très rapide.

Chapitre 4 : Circuits logiques

4.1.3 Opérateurs logiques et portes logiques

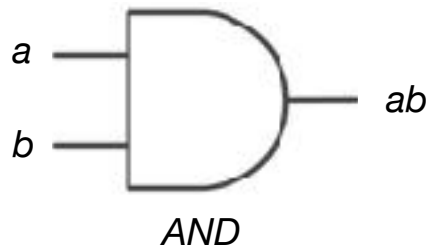
• NON / NOT

a	not
0	1
1	0



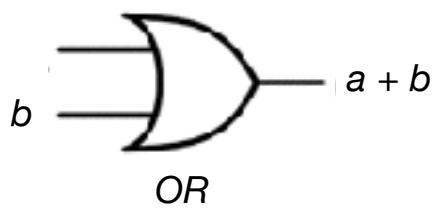
• ET / AND

a	b	and
0	0	0
0	1	0
1	0	0
1	1	1



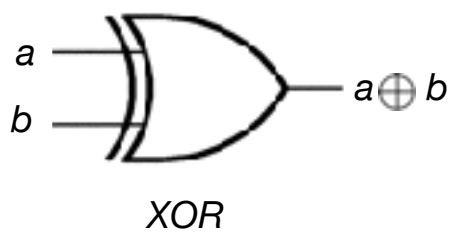
• OU / OR

a	b	or
0	0	0
0	1	1
1	0	1
1	1	1



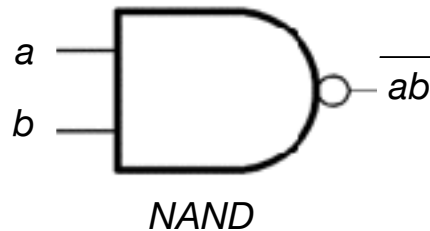
• OU EXCLUSIF / XOR

a	b	xor
0	0	0
0	1	1
1	0	1
1	1	0



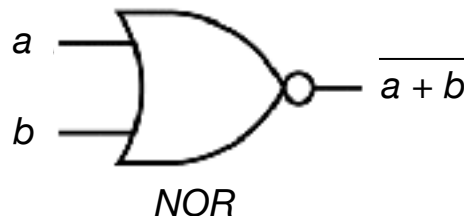
• NON-ET / NAND

a	b	nand
0	0	1
0	1	1
1	0	1
1	1	0



• NON-OU / NOR

a	b	nor
0	0	1
0	1	0
1	0	0
1	1	0



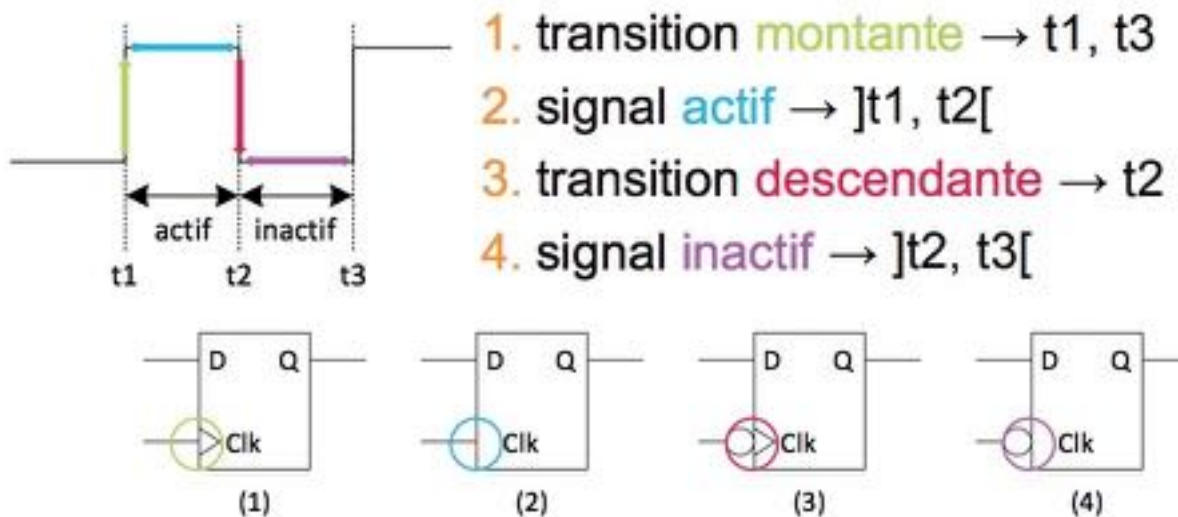
4.2 / 4.5 *Circuits combinatoires et séquentiels*

- **Circuit combinatoire** : est un circuit logique possédant un certain nombre d'entrées et de sorties, pour lequel la valeur des sorties à un instant donné est **uniquement déterminée par la valeur des entrées** à cet instant.
- **Circuit séquentiel** : est un circuit dans lequel les signaux de sortie à un instant donné ne dépendent pas uniquement des valeurs d'entrée du circuit à cet instant mais également de **l'historique du circuit**, c'est-à-dire de la séquence des transitions réalisées précédemment par le circuit. Ce résultat est obtenu par l'utilisation de **boucles de rétroaction** dans le circuit.
- **Multiplexeur, démultiplexeur, décodeur + schéma** : voir cours.

4.2 / 4.5 *Circuits asynchrones et synchrones*

- **Signal d'horloge** : sert à **synchroniser** le fonctionnement des composants électroniques sur une référence de temps.
- **Fréquence d'horloge** : temps qui sépare deux impulsions. Egalement appelé cycle d'horloge.
- **Circuit synchrone** : circuit dit **synchronisé** si son fonctionnement est contrôlé par un signal d'horloge.
- **Circuit asynchrone** : circuit dit **non-synchronisé** lorsque les signaux se propagent librement dans le circuit.

La modification d'état du circuit peut intervenir à 4 moments différents :



d

Chapitre 5 : L'unité centrale de traitement

5.2.4 Les modes d'adressage

- **Adressage absolu** : l'adresse effective de l'instruction suivante est spécifiée dans l'instruction.
- **Adressage relatif** : l'adresse effective de l'instruction suivante est obtenue à partir de l'adresse contenue dans le **compteur ordinal** (*pc*) en lui ajoutant/retranchant une certaine quantité spécifiée dans l'instruction.
- **Adressage indirect** : l'instruction indique dans quel registre se trouve l'adresse effective de l'instruction suivante.

Dans le cas de l'accès à une donnée :

- **Adressage immédiat** : la donnée est présente directement dans l'instruction. Il n'y a donc pas d'adressage à proprement parlé.
- **Adressage implicite** : l'opération utilise un registre qui est prédéfini par le processeur.
- **Adressage direct** : l'adresse fournie dans l'instruction indique où se trouve la donnée en mémoire.
- **Adressage indirect** : l'adresse fournie dans l'instruction indique l'adresse en mémoire où se trouve l'adresse de la donnée.
- **Adressage indexé** : l'adresse effective est obtenue en additionnant une valeur fournie dans l'instruction au contenu d'un registre d'index.

5.4 Améliorer les performances

- **Traitement vectoriel** : consiste à exécuter la même instruction sur plusieurs données simultanément.
- **Pipelining** : consiste à exécuter les instructions successives d'un même programme simultanément. L'idée est de se rapprocher du **travail à la chaîne**. On décompose une tâche complexe en une série de tâches plus simples pouvant être exécutées à la suite par des **unités autonomes**. Il permet de démarrer l'exécution de l'instruction suivante **sans attendre** que l'exécution de l'instruction précédente soit terminée. Un pipeline ne diminue pas le temps total nécessaire à la réalisation d'une seule tâche mais **augmente le débit** c'est-à-dire le **nombre de tâches réalisées par unité de temps**.

Limitations du pipelining :

- *Dépendance de données entre deux instructions* : il faut **retarder** artificiellement l'exécution de l'instruction 2.
- *Conflit de ressources* : lorsque deux étages du pipeline doivent accéder simultanément à une même ressource matérielle.
- *Branchement* : si une instruction de branchement conditionnel est exécutée, il faut attendre la condition de branchement soit évaluée.

Améliorations :

- **Data forwarding** : optimisation matérielle qui consiste à modifier le chemin de données du processeur. On crée un raccourci interne au chemin de données qui permet de réduire les retards de l'exécution.
- **Exécution dans le désordre** : consiste à modifier l'ordre des instructions d'un programme de manière à en optimiser l'exécution sans modifier le résultat final. Permet de réduire l'impact des dépendances de données ou des conflits de ressources.
- **Exécution spéculative** : essaye de prédire le résultat du branchement conditionnel et de poursuivre l'exécution du programme en fonction de cette prédiction.

5.5 Processeur vectoriel

Principe : l'objectif n'est pas d'exécuter plusieurs instructions simultanément comme le pipeline mais d'exécuter une **même instruction** sur plusieurs données simultanément. Efficace pour traiter des algorithmes mathématiques.

5.6 Architecture super scalaire

Principe : multiplie les unités fonctionnelles du chemin de données de manière à permettre l'exécution simultanée de plusieurs instructions sans provoquer de **conflit de ressources** mais reste **pénalisé** par les dépendances de données entre instructions et par les instructions de branchement.

5.7 Exceptions et interruptions

Exception : événement imprévu dont la source est **interne** au processeur.
Exemple : instruction inconnue, erreur arithmétique, ...

Interruption : événement imprévu dont la source est **externe** au processeur.
Exemple : requête d'un périphérique E/S, problème matériel hors processeur.

Chapitre 6 : La hiérarchie des mémoires

6.1 Introduction

Nous avons besoin de mémoire pour stocker les instructions des programmes et leurs données.

Contraintes techniques :

- La capacité de stockage d'une mémoire, est toujours limitée.
- La vitesse d'accès à un élément en mémoire, dépend de la technologie utilisée.

—> Les mémoires très rapides utilisent des technologies coûteuses mais sont de faibles capacités.

—> Les mémoires de grande capacité utilisent des technologie peu coûteuses mais sont plus lentes.

—> Il faut donc trouver un compromis entre le coût, la capacité et la performance.

Hiérarchie des mémoires : hiérarchie dans laquelle chaque niveau peut échanger des données avec les niveaux adjacents. Cela permet de créer l'illusion d'une mémoire unique à la capacité quasi illimitée.

Registres : Petite mémoire très rapide situé dans le processeur qui permet de stocker les données qu'il utilise.

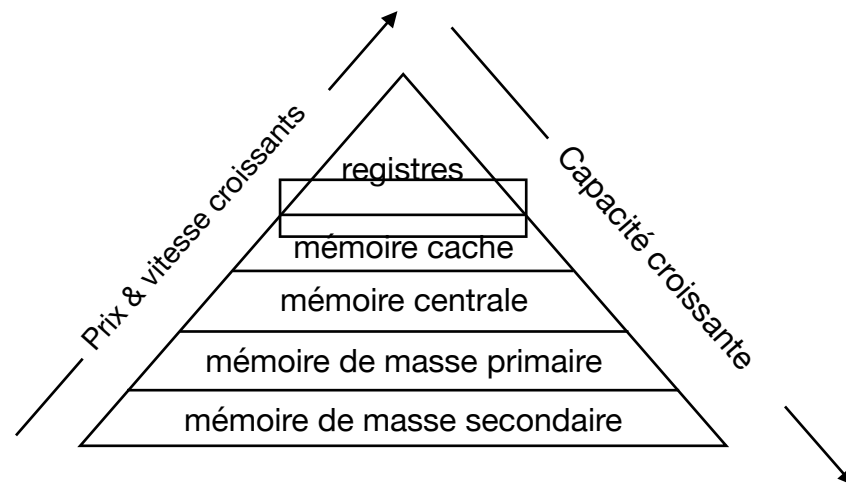
Mémoire cache (antémémoire) : Mémoire très rapide de faible capacité intégrée au processeur afin de réduire les temps d'accès. Sert de zone tampon entre le processeur et mémoire centrale.

Mémoire centrale : Mémoire principale rapide qui stocke les programmes et données, mais moins rapide car technologie moins coûteuse —> plus grande capacité. Meilleur temps d'accès car elle n'est pas intégrée au processeur.

Mémoire de masse primaire : Constituée de disques magnétiques ou SSD (Solid-State Drive). Stock de grands volumes de données **persistants** contrairement à la mémoire centrale. Offre un accès permanent et plutôt rapide aux données stockées.

Mémoire de masse secondaire : Utilisée pour stocker de très grands volumes de données de façon **persistante** à un très faible coût comme archiver des données ou effectuer des copies de sauvegardes. On la retrouve dans les CD, BluRay, DVD, ... L'accès aux données est très lent.

Schéma :



6.2 Caractéristiques des mémoires

- Les données stockées dans une mémoire sont organisées en entités

Bit : Information binaire élémentaire (0 ou 1).

Octet (byte) : Ensemble de 8 bits, constitue souvent la plus petite quantité d'information à laquelle il est possible d'accéder.

Mot (word) : Ensemble de plusieurs octets (souvent 32 ou 64). La taille d'un mot mémoire correspond habituellement à la taille « standard » d'une donnée transférée.

- Pour les mémoires de masses

Enregistrement (record) : Unité d'information dans un fichier ou une base de données.

Fichier (file) : Un ensemble d'enregistrements.

Volume (volume) : Zone de stockage accessible comme un seul bloc et contenant des fichiers.

- Caractéristique des mémoires

Adresse : Valeur numériques qui permet d'identifier l'emplacement d'une unité d'information dans la mémoire. La taille d'une adresse détermine le nombre d'éléments adressables individuellement.

Capacité : C'est le volume d'information, généralement exprimé en octets, que l'on peut stocker dans une mémoire. Exemple : 2 gigaoctets.

Temps d'accès : C'est le temps total nécessaire pour réaliser une opération de lecture ou écriture en mémoire.

Cycle mémoire : C'est le temps minimal qui doit s'écouler entre deux accès successifs à la mémoire.

Débit : Quantité d'information lues ou écrites par seconde.

Volatilité : C'est la capacité (ou non) d'une mémoire à conserver de façon permanente les informations qui y sont stockées en l'absence d'énergie électrique.

Type d'accès : Désigne la manière dont il est possible d'accéder à une information particulière :

- *Accès direct* : Accès direct à l'information sur base de son adresse, c'est l'accès le plus rapide.
- *Accès séquentiel* : Lecture des données stockées en commençant par la première jusqu'à celle désirée, accès assez lent.
- *Accès semi-séquentiel* : Combinaison des deux autres accès; On accède directement au bloc d'information puis on effectue une recherche de donnée séquentielle.
- *Accès par le contenu (mémoire associative)* : Organise l'information en associant chaque valeur mémorisée à une clé d'identification unique. Retrouver une information consiste donc à rechercher la clé dans la mémoire associative puis à renvoyer la valeur associée à cette clé.

- La disponibilité des informations

Le stockage en ligne (online) : Les informations sont directement disponibles, stockées sur un dispositif accessible en permanence soit raccordé à l'ordinateur soit via le réseau.

Le stockage quasi en ligne (near-line) : Les informations font parties d'une bibliothèque de type [jukebox](#) contenant un grand nombre de supports de stockage et un / plusieurs dispositif(s) de lecture.

Le stockage hors ligne (offline) : Les supports contenant les informations sont stockés dans une bibliothèque non automatisée. Accès sécurisé ou non (confidentialité données). Elle peut être localisée sur un autre site (plan catastrophe) et aménagée pour maximiser sa longévité.

6.3 Mémoire centrale

Tous les circuits mémoires actuels sont basés sur une technologie à **base semi-conducteurs**. La capacité de la mémoire centrale est souvent en gigaoctets ou téraoctets.

La mémoire centrale offre un accès direct aux données stockées sur base d'une adresse. On parle aussi d'accès aléatoire ou **RAM** (**R**andom **A**ccess **M**emory).

Toutes les mémoires offrent un accès en lecture (sinon inutile) mais suivant la technologie employée, les cellules mémoire peuvent être écrites de nombreuses fois, une seule ou pas du tout. Plusieurs types de mémoires différentes :

RWM (*Read-Write Memory*) : Mémoire dont les cellules peuvent être lues et écrites de nombreuses fois. Dénommé **mémoire vive** ou parfois *mémoire RAM*.

ROM (*Read-Only Memory*) : Mémoire dont les cellules peuvent être lues mais pas modifiées. Dénommée **mémoire morte**. Le contenu est déterminé définitivement par le fabricant.

PROM (*Programmable ROM*) : Mémoire morte programmable une seule fois par l'utilisateur et de manière irréversible.

EPROM (*Erasable Programmable ROM*) : Mémoire morte reprogrammable un certain nombre de fois. Nécessite une technique à l'ultraviolet afin de l'effacer.

EEPROM (*Electrically Erasable Programmable ROM*) : Même principe que l'EPROM mais effacement électrique, sélectif et plus rapide.

Deux grandes catégories de mémoires vives :

SRAM (Static RAM) : Mémoire vive *statique*. Cellule mémoire avec une structure similaire à celle d'une bascule D, technologie très rapide à faible consommation. Tant que la mémoire est alimentée, son contenu est conservé indéfiniment **sans manoeuvre particulière**. Temps d'accès en nanoseconde. Utilisée pour réaliser les registres du processeur et mémoire cache.

DRAM (Dynamic RAM) : Mémoire vive *dynamique*. Cellule mémoire avec 1 transistor et 1 condensateur qui se charge / décharge si il faut stocker 0 ou 1. L'information stockée s'altère au fil du temps. Même si le contenu de la mémoire ne se modifie pas, il faut **rafraichir régulièrement le contenu**. Plus complexe et plus lent que SRAM. Temps d'accès en quelques dizaines de nanoSec. Densité d'intégration 4x plus grande qu'une SRAM -> puce de très grande capacité à faible coût.

Plusieurs types de mémoires DRAM :

FPM : Fast Page Mode, **EDO** : Extended Data Output, **SDRAM** : Synchronous DRAM, **DDR** : Double Data Rate SDRAM.

Chaque nouvelle génération de DRAM améliore le **débit des transferts de données avec la mémoire**.

Circuit mémoire : Puce dont la capacité maximale dépend de la technologie utilisée. Pour avoir une mémoire centrale de grande taille, on utilise beaucoup de puces réparties sur des cartes mémoires où des barrettes reliées à la carte mère.

Barrette de mémoire : Petits circuits imprimés comportant généralement 8 ou 16 puces mémoire.

- **SIMM** (Single Inline Memory Module) : Dispose de 72 contacts et peut transférer 32 bits de données par cycle d'horloge.
- **DIMM** (Dual Inline Memory Module) : Dispo de 120 contacts sur chaque face du circuit (240 au total) et peut transférer 64 bits de données par cycle d'horloge.
- **SODIMM** (Small Outline DIMM) : Barrette DIMM mais moins encombrante, souvent utilisée dans les ordinateurs portables.

Le choix d'une configuration de mémoire a toujours pour objectif de maximiser le débit des transferts de données avec la mémoire. Certains contrôleurs imposent le transfert par bloc que par octet afin d'augmenter le débit.

Entrelacement : Technique qui consiste à répartir les mots mémoire successifs sur des barrettes mémoire différente -> accès mémoire sur le mot suivant sans attendre la fin du cycle.

6.4 Mémoire de masse

Objectif :

- Augmenter les performances -> vitesse lecture et écriture.
- Meilleure tolérance aux pannes grâce à la redondance des données.

On parle de « niveaux RAID » mais il n'y a aucune hiérarchie entre eux, les systèmes RAID les plus connus sont ceux du niveau 1 à 5.

RAID 0 :

- Regroupe plusieurs disques physiques pour créer un **volume de grande capacité**.
- Plusieurs disques physiques afin d'**améliorer les performances** du système.
- Lecture et écriture de gros fichiers de manière **circulaire** sur les disques.
- L'organisation des données doit être identique sur tous les disques.
- Si on utilise 'x' disque, la capacité total de la grappe est = à 'x' fois la capacité du plus petit disque.
- N'offre **aucune tolérance aux pannes**.
- **Diminue la fiabilité** globale du système car si 1 disque tombe en panne, perte de l'entièreté des données.

RAID 1 :

- Une grappe contient un nombre **pair** de disque.
- Capacité de 'x' fois la capacité du plus petit / 2.
- Lors de l'écriture, il faut écrire 2 fois, **disque primaire** et **disque miroir**.
- Ecriture soit simultanée (**duplexing**) soit en 2 temps (**mirroring**).
- **Performance** de lecture **doublée** car possible 2 secteurs en parallèle.
- Si un des disques tombe en panne, les **données restent disponibles**.

RAID 0 + 1 :

- Certains niveaux de RAID peuvent être combinés entre eux. Par exemple, une grappe RAID 0 peut être dupliquée en miroir, on parle alors de RAID 01 ou RAID 0 + 1.

Différentes architectures de stockage :

DAS (*Direct Attached Storage*) : Architecture standard où le système de stockage est **directement raccordé** au serveur.

NAS (*Networked Attached Storage*) : Système de stockage **connecté à un réseau**. Envoi d'une requête au serveur distant via protocole de gestion de fichiers standard et le serveur renvoie les données demandées.

SAN (*Storage Area Network*) : Ce n'est ni un système de stockage ni un serveur de données. C'est une manière de **partager des ressources** de stockages massives entre plusieurs serveurs via un **réseau de communication ultra-rapide** qui interconnecte serveurs et disques. L'accès aux données se fait par **bloc** (secteur).

Ces différentes architectures sont complémentaires, NAS et SAN utilisent la technologie RAID.

6.5 Mémoire cache (antémémoire)

Mémoire cache : Circuit mémoire très rapide (SRAM) mais de très faible capacité. Placé entre le processeur et la mémoire centrale. Elle a pour objectif de stocker les instructions et les données les plus utilisées afin de réduire au maximum le nombre d'accès à la mémoire centrale -> limiter l'impact de ces accès très lents.

Un processeur peut posséder plusieurs niveaux de mémoire cache. Chaque niveau communique exclusivement avec le niveau suivant ou précédent de la hiérarchie. Exemple : Si un CPU a besoin d'une donnée, il interroge la mémoire cache de nv. 1 Si la donnée n'est pas disponible, la mémoire cache de nv. 1 fait appel à celle de niv. 2, ...

Principe de localité :

Localité spatiale : Si un programme utilise une donnée située dans un emplacement mémoire, il est susceptible d'utiliser plus tard les données situées à proximité. Si un mot mémoire doit être placé en mémoire cache, alors on transfère TOUT le bloc auquel le mot appartient de sorte à ce que si le programme doit accéder au mot mémoire suivant ou précédent, il sera déjà dans la mémoire cache. Un tel bloc s'appelle *ligne de cache*.

Localité temporelle : Si un programme utilise une donnée, il est susceptible de la réutiliser dans un futur proche. Lorsqu'une donnée est utilisée, elle est placée en mémoire cache. Si la même donnée est utilisée à nouveau peu de temps après, elle est probablement toujours présente dans la mémoire cache.

Fonctionnement d'une mémoire cache :

Une mémoire cache est divisée en un certain nombre de lignes (bloc de données). Chaque ligne peut accueillir la copie d'un bloc de données de même taille provenant de la mémoire centrale.

Tâches pour la gestion de la mémoire :

- Si le processeur désire accéder en lecture ou écriture à une adresse mémoire, il faut déterminer si le bloc de données est déjà présent en mémoire ou pas.
- Déterminer dans quelle ligne sera stocké le bloc de données en cas de transfert mémoire centrale -> mémoire cache (correspondance / *mapping*).
- Quelle stratégie adopter en cas de saturation de la place dans les lignes.
- Si modification d'une données en mémoire cache, m.à.j en mémoire centrale.

Cache à correspondance directe (direct-mapped cache) :

Il faut choisir la ligne à utiliser dans la mémoire cache et se baser sur l'adresse du bloc en mémoire centrale.

cache miss : l'adresse accédée n'est pas en mémoire cache.

cache hit : l'adresse accédée est en mémoire cache.

Cache complètement associative (full-associative cache) :

Plus flexible que la mémoire cache à correspondance directe, un bloc de données peut être placé dans n'importe quelle ligne de la mémoire cache => flexibilité maximale.

Cache associative par ensemble (set-associative cache) :

Compromis entre la mémoire à correspondance directe et la mémoire complètement associative. La mémoire cache est divisée en plusieurs ensembles de N lignes de caches. L'index de l'ensemble dans lequel un bloc de données doit se trouver est déterminé au moyen d'une correspondance directe. Le bloc peut ensuite être placé dans n'importe laquelle des N lignes de cet ensemble (méthode associative).

Algorithmes de remplacement :

Il est nécessaire de garder une cohérence entre les données dans la mémoire cache et dans la mémoire de niveau supérieur.

Écriture immédiate (write-through) : répercuter immédiatement tout changement dans une mémoire cache de niveau supérieur.

- + très simple à mettre en oeuvre.
- peu efficace.
- écriture vers mémoire centrale => ralentit le processeur.

File d'attente (write buffer) :

- + mémoire cache continue à fonctionner à vitesse max.
- + données écrites au fur et à mesure en mémoire centrale.
- si file d'attente pleine, retour au cas précédent.

Écriture différée (write-back) : Les données sont écrites dans un premier temps uniquement dans la mémoire cache. Une ligne de cache modifiée est écrite en bloc dans la mémoire de niveau supérieur.

- + efficace car ne nécessite pas un accès vers la mémoire centrale.

6.5 Mémoire virtuelle

Mémoire virtuelle (virtual memory) :

- Facilite et sécurise le partage de la mémoire centrale entre plusieurs programmes.
- Donne l'illusion à un programme qu'il a accès à une mémoire d'une taille supérieure à celle de la mémoire centrale.

Adresse physique : représente une adresse précise en mémoire centrale, c'est-à-dire une cellule déterminée dans un circuit mémoire.

Adresse virtuelle : désigne un emplacement dans un espace d'adressage logique (virtuel).

La mémoire cache permet d'accéder rapidement à des portions de la mémoire centrale en stockant localement une partie des instructions et des données d'un programme. On peut imaginer que la **mémoire centrale** joue à son tour le rôle de mémoire cache par rapport à la **mémoire secondaire**. Afin de limiter les temps d'accès, les données sont transférées par bloc de 16 Ko appelé **pages**.

Chapitre 7 : Les entrées et sorties

7.1 Introduction

Un problème courant auquel nous devons faire face est l'extrême lenteur des périphériques d'E/S par rapport à la vitesse du CPU. Afin d'utiliser efficacement chaque périphérique doit être en mesure de réaliser de manière autonome des tâches +/- complexes et mettre en place un système de **signalisation** pour le CPU quand la tâche est terminée ou un quand un périph. a besoin de lui.

Deux systèmes existent :

- **Drapeaux** (flags) : le périphérique signale qu'il est prêt en positionnant un **drapeau** (bit) dans un registre de statut. Le processeur passe en revue les drapeaux afin de déterminer les disponibilités.
 - Pas très efficace car CPU doit tout le temps surveiller.
- **Interruption** (interrupts) : le périphérique envoie un signal (**interruption**) au CPU pour signaler qu'il est prêt. Le processeur interrompt l'exécution du programme en cours et exécute un programme de gestion spécial qui réalise l'E/S.
 - + approche la plus répandue.

7.2 Interruptions hiérarchisées

Le **système d'interruptions hiérarchisées** est un système mis en place dans les ordinateurs modernes. A chaque interruption est attribué une **priorité**, dans le cas où plusieurs périphériques E/S rentrent en conflit par exemple.

- Les interruptions de même priorité sont traitées par ordre d'arrivée.
- Le programme de traitement d'interruption peut lui même être interrompu par une interruption de priorité supérieure.
- Lorsque le traitement d'une interruption est terminée, le contrôle du CPU est attribué au programme en attente qui a la priorité la plus élevée.

On obtient donc une **cascade d'appels imbriqués**.

7.3 Contrôleur de périphérique

Contrôleur de périphérique : circuit électronique spécialisé qui permet de raccorder un équipement d'E/S à l'ordinateur.

Il existe une grande variété de périphériques E/S avec des caractéristiques très différentes => l'ordinateur impose une **interface standardisée** pour communiquer.

Le rôle premier du contrôleur est de réaliser la conversion entre l'interface propriétaire du périphérique et l'interface standardisée de l'ordinateur.

7.4 Accès direct à la mémoire (DMA)

DMA (*Direct Memory Access*) : dispositif qui permet à un périphérique d'échanger directement des données avec la mémoire centrale sans faire appel au CPU.

- *Fonctionnement* :

Le CPU démarre l'opération d'E/S en fournissant au contrôleur DMA toutes les informations nécessaires. Le DMA réalise le transfert de données et signale la fin de l'opération via une interruption.

Il n'est pas possible d'effectuer deux accès simultanés => CPU et DMA entrent en **conflit**. Le DMA est **prioritaire** sur le CPU. Le DMA obtient un accès exclusif à la mémoire pendant la durée du transfert, on appelle ça un **vole de cycles** (*le DMA vole des cycles d'accès à la mémoire au CPU*).

7.5 Canaux d'entrées/sorties

Canal d'entrées/sorties : processeur spécialisé programmable capable d'exécuter de véritables programmes d'E/S et d'enchaîner plusieurs opérations successives. Un canal d'E/S peut être dédié à un seul périphérique (**canal sélecteur**) ou partagé entre plusieurs (**canal multiplexé**).

7.6 Bus de communication

Bus : moyen de communication qui relie plusieurs composants ou équipements pour leur permettre d'échanger des informations. Il est composé d'un **canal de communication physique** et d'un **contrôleur**.

Bus interne : interconnecte différents composants au sein d'un ordinateur.

- *Bus système* : connecte le CPU à la mémoire centrale à très grande vitesse.
- *Bus d'E/S* : appelé aussi **bus d'extension**, permet de connecter des périphériques d'E/S ou des cartes d'extension.

Bus externe : permet de connecter des périphériques à un ordinateur, ou de connecter plusieurs ordinateurs entre eux.

- La transmission des informations sur un bus peut être **synchrone** ou **asynchrone**.
- Les données peuvent être transmises sur le bus un seul bit à la fois (**bus série**) ou plusieurs bits à la fois (**bus parallèle**).
- Des bus de différents types peuvent être interconnectés en utilisant un **pont**.

Deux principaux bus d'extension utilisés :

- **PCI** (*Peripheral Component Interconnect*) : Bus **parallèle** de 32 ou 64 bits partagé entre tous les périphériques connectés.
- **PCI Express** (*PCIe*) : Bus **série** qui transporte les données par paquet. PCI Express n'est pas vraiment un bus, il s'agit d'un ensemble de **liaisons** (*links*) **point-à-point** qui connectent les périphériques à un **noeud central** (*root complex*). La transmission des informations s'effectue en série (1 bit à la fois) sur des **lignes** (*lines*).
La norme PCI Express est conçue pour fonctionner avec l'ancienne norme PCI. Un **pont** réalise l'adaptation entre le bus PCI et le monde PCIe.

7.7 Les réseaux

Réseau informatique : prolongation naturelle d'un bus externe. Il permet à des équipements (nœuds du réseau) de communiquer au-delà de l'environnement immédiat de l'ordinateur.

Classement des réseaux par étendue :

Réseau personnel - PAN (*Personal Area Network*) :

- les nœuds se trouvent dans un voisinage immédiat de **quelques mètres** (clavier, imprimante, ...).

Réseau local - LAN (*Local Area Network*) :

- les nœuds se trouvent dans le même bâtiment ou dans des bâtiments voisins => distance de **quelques dizaines ou centaines de mètres**.

Réseau métropolitain - MAN (*Metropolitan Area Network*) :

- les nœuds se trouvent dans une même métropole => distance de **quelques à une dizaine de kilomètres**.

Réseau étendu - WAN (*Wide Area Network*) :

- les nœuds sont géographiquement très éloignés, **de quelques centaines à milliers de kilomètres**. *Internet* est un WAN à l'échelle mondiale.

WLAN (*Wireless LAN*) : transmission de type Wifi.

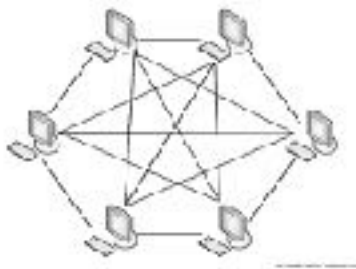
WPAN (*Wireless PAN*) : transmission de type Bluetooth, Wifi, ...

Les méthodes de diffusion :

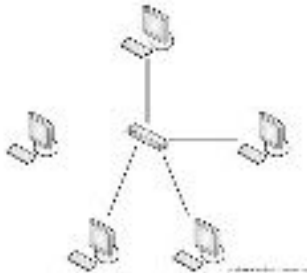
- **Point-à-point** (*unicast*) : chaque utilisateur connecté au réseau est identifié par une adresse. L'expéditeur et le destinataire du message ne sont pas spécialement connecté entre eux. Même principe qu'un [service postal](#).
- **Diffusion** (*broadcast*) : envoyer un message à [tous](#) les utilisateurs. Même principe que la [radio](#) par exemple.
- **Multidiffusion** (*multicast*) : envoyer un message à un groupe [limité](#) d'utilisateur. Même principe qu'une [vidéoconférence](#) (Skype, ...).

Topologie des réseaux :

- **Réseau maillé**



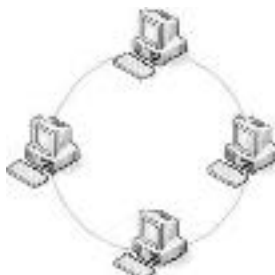
- **Réseau en étoile**



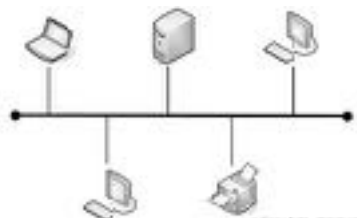
- **Réseau en arbre**



- **Réseau en boucle**



- **Réseau en bus**



Chapitre 8 : Le système d'exploitation

8.1 Introduction

Système d'exploitation (*Operating System - OS*): est le maillon indispensable situé entre les couches **matérielles** (partie système) et les couches **logicielles** (partie utilisateur) de l'ordinateur.

Permet de (liste non-exhaustive) :

- Assure la gestion des ressources matérielles.
- Organise l'exécution des programmes.
- Prend en charge la gestion des périphériques E/S.
- Organise le stockage d'information
- ...

8.2 Le noyau

Noyau (*kernel*) : portion du système d'exploitation qui se charge de l'interface entre le logiciel et le matériel au **niveau le plus bas**.

Le noyau s'occupe de :

- Allocation du/des CPU(s) aux différents processus.
- La gestion des exceptions et des interruptions.
- L'accès de bas-niveau à la mémoire centrale.
- L'accès de bas-niveau aux périphériques d'E/S.

Ces tâches sont le fonctionnement de l'ordinateur => c'est la seule portion de l'OS qui doit **résider en permanence en mémoire centrale**. Le noyau est généralement codé en **assembleur** pour garantir que son exécution soit la plus **rapide** possible.

8.3 Systèmes multitâches

Système multiprogrammé : système dans lequel plusieurs programmes sont présents simultanément en mémoire centrale. Généralement l'OS permet d'exécuter plusieurs programmes simultanément, on parle alors d'un système d'exploitation **multitâches**.

Système multitâches : le processeur alloue une partie de son temps en **alternance** aux différents programmes qu'il doit exécuter. L'alternance rapide donne cette impression d'exécution simultanée.

Processus : exécution d'un programme (ensemble d'instructions) par un processeur.

Différentes techniques d'exécution simultanée :

Multiprogrammation : (*multiprogramming*) : allouer le CPU à l'exécution d'un programme dès qu'une opération d'E/S était réalisée par le programme.

- Aucune garantie de partage équitable.
- Un programme peut monopoliser le CPU.

Multitâche coopératif : (*nonpreemptive multitasking*) : première évolution vers un **partage de temps** (*time-sharing*) plus équitable. Chaque programme est conçu de manière à rendre **périodiquement** la main au système d'exploitation et donc laisser d'autres programmes s'exécuter.

- L'OS n'as pas de contrôle sur ce qu'il se passe => pas d'optimisation du CPU en fonction des besoins en ressources matérielles.
- Les programmes doivent coopérer de manière équitable.
- Un bug (ex : boucle infinie) peut bloquer le système entier.

Multitâche préemptif (*preemptive multitasking*) : permet de garantir une répartition équitable des ressources du CPU. C'est l'OS qui gère l'allocation du CPU aux différents processus.

Le passage d'un processus à un autre s'effectue sur base :

- d'événements systèmes.
- durée d'exécution maximale.
- de la priorité.

Le système d'exploitation comporte un processus appelé **planificateur** (*scheduler*) qui réalise l'ordonnancement des tâches. Ces tâches sont placées en file d'attente.

Un second processus appelé **allocateur** (*dispatcher*) est chargé de répartir le temps du CPU entre les différents processus de la file d'attente.

Changement de contexte (*context switching*) : lorsque le processeur passe d'un processus à un autre. Il faut sauvegarder **l'état d'exécution** du processus interrompu => son **contexte** afin de reprendre son exécution ultérieurement. C'est l'allocateur qui se charge de sauvegarder le contexte du processus dans les registres et de rétablir le contexte d'exécution ultérieurement.

8.4 Gestion de la mémoire

Lorsqu'un seul programme à la fois peut se trouver en mémoire centrale, la mémoire est utilisée uniquement par l'OS et le programme à exécuter. L'OS utilise la **mémoire basse** càd celle qui démarre à **l'adresse 0** et le programme utilise la **mémoire haute** càd celle située au-delà de la partie occupée par l'OS.

Dans un système multiprogrammé c'est plus compliqué car plusieurs programmes doivent coexister en mémoire, la taille de chaque programme est variable et peut changer en cours d'exécution et un programme ne doit pas empiéter sur la mémoire allouée à un autre programme.

Plusieurs techniques sont utilisées :

1. Partitions fixes : les zones mémoires ont une taille définie à l'avance par l'OS. Toutes les partitions n'ont pas la même taille mais un programme ne peut être chargé en mémoire que si il y a suffisamment de place et si il n'utilise pas toute la mémoire qui lui est alloué, il y a un gaspillage important de la mémoire centrale.

2. Partitions de tailles variables : la taille d'une partition correspond exactement à la quantité de mémoire demandée par un programme. Le problème est que la taille de la partition est fixée définitivement lors du chargement du programme et il peut également y avoir un gaspillage. De plus, la mémoire libre va avoir tendance à se fragmenter et donc diminution de la place => il faut régulièrement **compacter la mémoire** c'àd déplacer les partition en mémoire pour regrouper les zones utilisées et recréer de grandes zones libres.

Réallocation : lorsqu'un programme est déplacé en mémoire, cela entraîne une modification des adresses référencées => utilisation de **l'adressage relatif** (ou indexé) : un registre prédéfini contient l'adresse de début de programme et toutes les autres adresses référencées. Il faut simplement modifier l'adresse de base dans le registre.

3. Segments : diviser le programme en portions +/- petites qui contiennent seulement une partie de l'instruction ou des données du programme => la fragmentation est moins problématique et diminution de la fréquence des réallocations. Un programme peut également libérer des segments durant son exécution.

Protection de la mémoire :

- Chaque programme utilise le mode **d'adressage relatif** => il n'utilise pas des adresses mémoire absolues mais calcule toute adresse effective relativement au contenu d'un **registre de base**.
- Le registre de base contient l'adresse de début de la partition en mémoire.
- Un deuxième registre, appelé **registre de borne**, contient quant à lui l'adresse supérieure de la partition.
- Seul l'OS est autorisé à modifier le contenu des registres de base.
- Lorsqu'un programme s'exécute, un dispositif du processeur vérifie que toute adresse référencée par le programme correspond bien à la zone mémoire allouée.

Chapitre 9 : Les architectures à processeurs multiples

9.1 Introduction

Parallélisme : capacité à effectuer plusieurs tâches simultanément.

Processeur spécialisé : conçu pour réaliser un type de tâche bien défini (ex : cryptage, compression de données, ...) . Il est alors possible d'optimiser spécifiquement son architecture en fonction des tâches à réaliser.

9.2 Différentes formes de parallélisme

Parallélisme interne : consiste à exécuter plusieurs tâches simultanément au sein d'un seul processeur.

Techniques :

- pipelining.
- traitement vectoriel.
- architectures super scalaires.
- multitâche.

Parallélisme externe : plusieurs processeurs se répartissent les tâches en collaborant de manière plus ou moins étroite.

Techniques :

- *Processeurs à coeurs (core) multiples* : chaque coeur est un processeur à part entière. Le fait de les regrouper au sein d'un même circuit intégré permet une communication **très rapide**. Cela permet à plusieurs coeurs de collaborer de manière très efficace à l'exécution d'une **tâche commune**.
- Plusieurs processeurs dans un seul ordinateur (partage de la mémoire centrale uniquement).
- Plusieurs ordinateurs interconnectés par un réseau très rapide : **grappe** de serveurs, **grille informatique**.

Parallélisme transparent (point de vue utilisateur) : l'utilisateur n'a pas à se préoccuper de la manière dont les tâches sont exécutées. L'OS organise l'exécution simultanée et le compilateur optimise le code du programme. Le processeur maximise l'exécution du programme grâce au pipelining et à la mémoire cache.

Parallélisme explicite : l'utilisateur doit concevoir son programme de manière spécifique :

- Décomposer le programme en sous-tâches susceptibles d'être exécutées simultanément.
- Utiliser les instructions spécifiques d'un processeur vectoriel.
- Concevoir des algorithmes parallèles.
- Organiser les données par petits morceaux et les traiter simultanément.

9.3 Classification des systèmes parallèles

SISD (*Single Instruction, Single Data stream*) : c'est l'ordinateur classique basé sur l'architecture de Von Neumann. Il exécute une seule instruction et produit un seul résultat à la fois.

SIMD (*Single Instruction, Multiple Data streams*) : l'unité de contrôle va chercher les instructions du programme *une à la fois* mais dispose de plusieurs unités de calcul => exécute plusieurs instructions en parallèle.

MISD (*Multiple Instructions, Single Data stream*) : architecture qui peut appliquer simultanément *plusieurs instructions à une même donnée*. Pas d'ordinateur connu.

MIMD (*Multiple Instructions, Multiple Data streams*) : contient toutes les architectures composées de plusieurs processeurs travaillant en accord au sein d'un même système.

- *multi-coeurs / multi-processeurs* : caractérisé par un niveau de *couplage élevé*. Les différents processeurs sont connectés à un bus commun et partagent la mémoire centrale voire la mémoire cache pour les systèmes multi-coeurs (mémoire partagée).
- *multi-ordinateurs* : caractérisé par un couplage faible. les différents ordinateurs possèdent leur propre mémoire (mémoire distribuée). Ils sont connectés au moyen d'un réseau de communication propriétaire à très grande vitesse (processeurs massivement parallèles).